Reconciliation Exception Classification Review

Workspace review date: April 6, 2026

Scope traced end-to-end: parsing, normalization, matching, exception classification, summary generation, workbook export, API delivery, and web reporting

Executive Summary

The project is already capable of actionable reconciliation exception classification. It no longer relies on a user-facing generic unmatched bucket. The strongest parts of the implementation are the shared taxonomy, deterministic rule flow, detailed rule traces, category-specific next actions, and end-to-end surfacing through API, workbook export, and UI. The main remaining weaknesses are architectural rather than taxonomic: grouped reasoning still depends on bounded residual heuristics, reviewer decisions are not persisted, and some top-level docs still undersell the current capability.

Runtime Path Reviewed

1. Workbook parsing in `packages/core/src/parsers/excelParser.ts`.
2. Normalization in `packages/core/src/normalization/transactions.ts`.
3. Candidate ranking, duplicate detection, exact consumption, and grouped reservation in `packages/core/src/matching/engine.ts` and `packages/core/src/matching/groupedReservations.ts`.
4. Residual and review-driven exception classification in `packages/core/src/reporting/exceptionClassifier.ts`.
5. Summary metrics in `packages/core/src/reporting/summary.ts`.
6. Workbook surfacing in `packages/core/src/export/workbookExporter.ts`.
7. API exposure in `apps/api/src/services/reconciliationService.ts` and `apps/api/src/app.ts`.
8. UI reporting in `apps/web/src/App.tsx` and `apps/web/src/components/ExceptionTable.tsx`.

Current-State Findings

Verified Findings

- The shared domain model explicitly defines the implemented exception taxonomy in `packages/domain/src/enums.ts` and accountant-facing labels, messages, and next actions in `packages/domain/src/messages.ts`.
- The parser preserves original file name, worksheet name, line number, and raw source values, which supports downstream auditability.
- The normalizer deterministically parses dates and signed amounts, tracks data-quality issues, and preserves normalized token sets used by matching logic.
- The matcher is rule-based. It distinguishes exact matches, reference-based matches, description variance, amount mismatch, date variance, timing-oriented review candidates, duplicate rows, and ambiguous review candidates.
- The classifier converts residual and review-driven outcomes into explicit categories rather than a generic user-facing unmatched bucket.

- Final exceptions preserve rule traces, evidence, recommended actions, likely matching references, grouped counterpart arrays, and prior lineage traces where applicable.
- The workbook export includes category breakdowns, an Error Report, counterparts, and rule lineage instead of dumping flat unmatched rows.
- The API returns full typed exception objects and summary counts, and the web UI renders category filters, rule names, evidence, next actions, counterparts, and lineage.
- Regression coverage is broad across core, API, and UI, including all-exception fixtures, precedence checks, grouped classifications, stale checks, miscoding, amount mismatch, ambiguity, and config overrides.

Verified Gaps

- **High:** Grouped residual reasoning is still bounded by `groupedCandidatePoolSize` and `groupedMatchMaxSize`. This means larger or denser settlement patterns can still fall back to missing-item categories.
- **Medium:** Reviewer resolution is surfaced, but not persisted as a durable approval or override workflow.
- **Medium:** Top-level docs such as `README.md` and `docs/test-strategy.md` still summarize a narrower rule set than the current implementation actually supports.

Inferred Recommendations

- The next improvement should not be a taxonomy rewrite. The code already has one.
- The next architectural step should be to make grouped one-to-many and many-to-one reasoning a first-class matching model rather than a residual heuristic.
- The next operational step should be to persist reviewer decisions so probable and ambiguous review outcomes become part of the audit trail, not just an in-memory report artifact.

Classification Coverage Matrix

Classification	Supported?	Evidence in Code	Gap	Recommendation
Timing difference	Yes	Explicit residual timing stage and typed category in classifier, summary, tests, and UI/export surfaces.	Depends on timing window and kind-compatibility heuristics.	Add more adversarial transfer and month-end timing regressions.
Missing in bank	Yes	Explicit fallback category `MissingFromBank` with professional messaging and next actions.	Can still be a fallback when richer grouped context is missed.	Preserve as terminal fallback, but reduce false fallback by improving grouped reasoning.
Missing in QBO	Yes	Explicit fallback category `MissingFromQBO` with rule trace and next actions.	Same residual-fallback limit as above.	Keep as explicit fallback; do not dilute with generic unmatched reporting.

Classification	Supported?	Evidence in Code	Gap	Recommendation
Duplicate in bank	Yes	Same-source duplicate detection in matcher and dedicated downstream category.	No major gap observed.	Add reversal-pattern regressions if bank feeds commonly include them.
Duplicate in QBO	Yes	Dedicated duplicate stage and category-specific messaging.	No major gap observed.	Add more bookkeeping-specific duplicate fixtures.
Amount variance	Yes	Implemented as `AmountMismatch` from matcher through summary, API, and UI.	Label uses mismatch rather than variance.	Change the display label only if finance users prefer the word variance.
Date variance	Yes	Extended-window date rule exists and is surfaced separately from timing difference.	Some cases are intentionally promoted to timing difference or miscoding.	Document the promotion path more clearly in user docs.
Likely miscoding	Yes	Residual miscoding stage with counterpart traces, next actions, and regression coverage.	Still heuristic, based on same amount plus conflicting context.	Add more vendor, account, and memo-conflict fixtures.
Likely split/combine issue	Yes	Grouped residual split/combine category with grouped evidence and counterpart arrays.	Still bounded residual logic.	Promote grouped reasoning into a first-class grouped match model.
Stale outstanding check	Yes	Explicit stale-check stage plus configurable aging threshold.	Depends on inferred check-like behavior and reconciliation anchor date.	Add void and reissue scenario coverage.
Deposit grouping issue	Yes	Dedicated grouped deposit residual category with grouped evidence and counterpart arrays.	Same bounded grouped-residual limit.	Same structural recommendation as split/combine.

Logic and Architecture Assessment

The architecture is already strong enough to support detailed exception classification. The core flow is deterministic, the domain model is explicit, and the matcher-classifier handoff is intentionally staged. Exact matches are consumed in the matcher; advisory one-to-one review results remain visible; grouped, stale, miscoding, timing, and missing-item classifications are finalized in the reporting layer against the residual pool.

The main architectural weakness is grouped reasoning. The code now reserves grouped candidates before advisory one-to-one review, which is a material improvement, but grouped outcomes are still inferred from bounded residual search rather than generated as native grouped match results. That is the primary place where the system can still miss a more precise classification and fall back to missing-from-source outcomes.

Reporting and Auditability Assessment

The current reporting model is accountant-facing and substantially more actionable than a flat unmatched report. Each exception can include category-specific wording, severity, rule name, reason, evidence, next actions, counterpart references, and lineage from prior rule stages. Those details are preserved across domain types, workbook export, API output, and web UI rendering.

The main remaining auditability gap is workflow persistence. Reviewer-facing items are surfaced, but reviewer decisions are not yet stored as durable state. The report is explainable and reproducible from the same inputs and config, but the platform is not yet a full reviewed-state reconciliation system.

Prioritized Recommendations

Immediate Improvements

Recommendation 1: Align top-level documentation with the implemented taxonomy and reporting model

The codebase now supports a broader and more accountant-facing classification model than the top-level docs currently describe.

Problem observed

`README.md` and `docs/test-strategy.md` still summarize a narrower rule set than the implementation now exposes through the domain model, API responses, workbook export, and UI.

Why it matters

Documentation now understates the product. That creates avoidable confusion during onboarding, stakeholder review, and implementation planning because readers can miss that grouped issues, miscoding, stale checks, lineage, and counterpart tracing already exist.

Severity

Low

Implementation complexity

Low

Recommended solution

Update the README and test strategy docs to explicitly describe the current exception taxonomy, grouped residual categories, recommended-next-action support, rule lineage, and counterpart traceability. The docs should match what the API tests and workbook export already prove.

Impacted files/modules

`README.md`, `docs/test-strategy.md`, optionally `docs/api.md` and `docs/reconciliation-rules.md`.

Change type

Documentation

Suggested acceptance criteria
A new reader can identify all currently implemented exception categories and understand that exceptions surface with reasons, evidence, next actions, counterparts, and lineage without needing to inspect the source.

Recommendation 2: Remove the remaining user-facing implication of a generic unmatched bucket

The implementation does not use a final generic exception bucket, but some summary-oriented terminology still suggests that it might.

Problem observed
Internal state names such as `unmatchedBankTransactionIds`, `unmatchedQboTransactionIds`, and summary fields such as `unmatchedItems` are reasonable internally, but can read like a generic classification model when surfaced or paraphrased in reporting.

Why it matters
The product goal is to move away from catch-all unmatched reporting. If user-facing copy still centers the word unmatched, the reporting model can appear weaker than it is.

Severity
Low

Implementation complexity
Low

Recommended solution
Keep the internal matcher vocabulary if it remains useful, but update user-facing summary labels and explanatory text so they emphasize classified follow-up items rather than unmatched items.

Impacted files/modules
`packages/core/src/reporting/summary.ts`, `packages/core/src/export/workbookExporter.ts`, `apps/web/src/App.tsx`, optionally shared summary type names in `packages/domain/src/types.ts`.

Change type
UI-reporting

Suggested acceptance criteria
No user-facing screen or workbook tab suggests that exceptions are being left in a generic unmatched state when the final detail set is already categorized.

Near-Term Enhancements

Recommendation 3: Persist reviewer decisions for probable and ambiguous review items

The application already separates exact matches from review-oriented outcomes, but reviewer resolution is still transient.

Problem observed

Probable matches and ambiguous review items are exposed through the run payload and rendered in the UI, but the current repository implementation stores only the run snapshot and does not persist reviewer approvals, rejections, or reclassifications as durable workflow state.

Why it matters
This is the main remaining operational gap between a strong reporting engine and a complete reconciliation review workflow. Without persisted reviewer decisions, the audit trail stops at suggested outcomes rather than final reviewed disposition.

Severity
Medium

Implementation complexity
Medium

Recommended solution
Add a reviewer-decision model keyed to run ID and transaction or match identifiers. Support at least approve, reject, and manual reclassify actions, and return those decisions through the API so the UI and exported artifacts can show reviewed state explicitly.

Impacted files/modules
`apps/api/src/services/runRepository.ts`, `apps/api/src/app.ts`, `packages/domain/src/types.ts`, `apps/web/src/App.tsx`, and any future persistence-backed repository implementation.

Change type
Architectural

Suggested acceptance criteria
A reviewer can resolve a probable or ambiguous item, reload the latest run, and see the same reviewed outcome returned by the API and rendered in the UI. The decision history should remain attributable to the run and related transaction set.

Recommendation 4: Expand grouped-settlement regression coverage where heuristics compete

The grouped classification logic is materially better than a generic missing-item fallback, but it is also the most heuristic-sensitive part of the system.

Problem observed
The existing tests cover grouped split/combine and deposit grouping paths well, but there is still limited dense coverage for scenarios where grouped residuals, timing differences, miscoding, and advisory one-to-one review suggestions all compete for the same pool.

Why it matters
This is the part of the implementation most likely to regress as real-world settlement patterns become denser or more irregular. If it regresses, the system will not become generic, but it may misclassify richer grouped cases as missing items.

Severity
Medium

Implementation complexity
Medium

Recommended solution
Add fixtures that force competition between grouped reservations, advisory one-to-one review candidates, stale-check logic, and miscoding or timing promotions. Include cases with overlapping grouped candidates and denser same-amount pools.

Impacted files/modules
`packages/core/src/reconcile.test.ts`, `packages/core/src/reporting/exceptionClassifier.test.ts`, and fixture builders in `packages/test-fixtures`.

Change type
Test coverage

Suggested acceptance criteria
New regression fixtures prove that grouped classifications remain stable when competing heuristics are present and that fallback to missing-item categories only occurs when the grouped evidence really is insufficient.

Structural Redesign Items

Recommendation 5: Promote grouped one-to-many and many-to-one reasoning into a first-class matching model

This is the highest-value structural improvement remaining after the taxonomy, lineage, and reporting upgrades.

Problem observed
Grouped classifications are still derived from bounded residual search and reservation logic. That is a significant improvement over a pure unmatched fallback, but it means grouped outcomes are inferred after one-to-one candidate ranking rather than represented as native grouped match candidates and grouped review results.

Why it matters
Grouped deposits and split/combine patterns are a core reconciliation reality, not an edge-case cleanup step. As long as grouped reasoning is residual-only, the system can still miss the best explanation for denser settlement patterns and fall back to less specific missing-from-source outcomes.

Severity
High

Implementation complexity
High

Recommended solution
Introduce grouped match candidates and grouped match results as first-class concepts in the matching layer. Let the matcher generate, score, and surface grouped proposals directly, then let the classifier translate reviewed or unresolved grouped proposals into accountant-facing categories with full grouped context.

Impacted files/modules
`packages/core/src/matching/engine.ts`, `packages/core/src/matching/groupedReservations.ts`,

`packages/core/src/reporting/exceptionClassifier.ts`, `packages/domain/src/types.ts`, and downstream API/UI/export surfaces that consume match-result shapes.

Change type
Architectural

Suggested acceptance criteria
The engine can emit grouped candidate results directly, reviewers can see grouped proposals before fallback classification, and grouped categories no longer depend primarily on residual reconstruction bounded by small pool-size heuristics.

Suggested Decision Order

1. Data-quality gate.
2. Same-source duplicate detection.
3. Exact one-to-one matching.
4. First-class grouped candidate generation.
5. Advisory one-to-one review outcomes such as amount mismatch, reference variance, description variance, and ambiguity.
6. Residual promotions such as timing difference and likely miscoding.
7. Residual grouped fallback.
8. Missing-from-source fallback.
9. Aged check override for stale outstanding checks.

Example Improved Output

Category	Source	File / Line	Counterparts	Rule	Reason	Next Action
Timing Difference	Bank	bank.xlsx / 18	qbo.xlsx / 21	Residual Timing Difference	Same amount and timing-compatible counterpart remained unmatched inside the extended timing window.	Carry as timing support and monitor the next cycle.
Missing from QBO	Bank	bank.xlsx / 5	None	Missing from QBO	No QBO-side transaction remained after exact, review, and variance	Confirm whether the bank activity was booked to the correct

Category	Source	File / Line	Counterparts	Rule	Reason	Next Action
					matching rules.	QBO account and period.
Deposit Grouping Issue	QBO	qbo.xlsx / 12	bank.xlsx / 12 plus grouped receipt members	Grouped Deposit Residual	Residual unmatched deposits form a grouped receipt pattern rather than independent missing items.	Compare the bank deposit to batch support and document the grouped relationship.

This review reflects the current implementation state observed directly from source, tests, documentation, API shapes, and web/reporting surfaces in the workspace.